
Software Requirements Specification

for

Restaurant Reservation System

Version 1.0

Prepared by

Group Name: Group 17

Nareen Reddy	SE23UCSE120	se23ucse120@mahindrauniversity.edu.in
Pranav Akkina	SE23UCSE144	se23ucse144@mahindrauniversity.edu.in
Aryan Goud	SE23UARI038	se23uari038@mahindrauniversity.edu.in
Charith Leburu	SE23UARI074	se23uari074@mahindrauniversity.edu.in
Shanvi Rishi	SE23UARI136	se23uari136@mahindrauniversity.edu.in
Sreethan Reddy	SE23UCSE138	se23ucse138@mahindrauniversity.edu.in
Swetha Burra	SE23UCSE170	se23ucse170@mahindrauniversity.edu.in
Sejal Nadimatla	SE23UARI209	se23ucse209@mahindrauniversity.edu.in

Instructor: <place your instructor's name here>

Course: <place your course name here>

Lab Section: <place your lab section here>

Teaching Assistant: <place your TA's name here>

Date: <place the date of submission here>

CONTENTS.....	II
REVISIONS.....	II
1 INTRODUCTION.....	1
1.1 DOCUMENT PURPOSE.....	1
1.2 PRODUCT SCOPE.....	1
1.3 INTENDED AUDIENCE AND DOCUMENT OVERVIEW.....	1
1.4 DEFINITIONS, ACRONYMS AND ABBREVIATIONS.....	1
1.5 DOCUMENT CONVENTIONS.....	1
1.6 REFERENCES AND ACKNOWLEDGMENTS.....	2
2 OVERALL DESCRIPTION.....	2
2.1 PRODUCT OVERVIEW.....	2
2.2 PRODUCT FUNCTIONALITY.....	3
2.3 DESIGN AND IMPLEMENTATION CONSTRAINTS.....	3
2.4 ASSUMPTIONS AND DEPENDENCIES.....	3
3 SPECIFIC REQUIREMENTS.....	4
3.1 EXTERNAL INTERFACE REQUIREMENTS.....	4
3.2 FUNCTIONAL REQUIREMENTS.....	4
3.3 USE CASE MODEL.....	5
4 OTHER NON-FUNCTIONAL REQUIREMENTS.....	6
4.1 PERFORMANCE REQUIREMENTS.....	6
4.2 SAFETY AND SECURITY REQUIREMENTS.....	6
4.3 SOFTWARE QUALITY ATTRIBUTES.....	6
5 OTHER REQUIREMENTS.....	7
APPENDIX A – DATA DICTIONARY.....	8
APPENDIX B - GROUP LOG.....	9

Revisions

Version	Primary Author(s)	Description of Version	Date Completed
Draft Type and Number	Full Name	Information about the revision. This table does not need to be filled in whenever a document is touched, only when the version is being upgraded.	00/00/00

1 Introduction

The Restaurant Reservation System is a web-based application designed to help users easily discover restaurants and make table reservations online. The system allows customers to search for restaurants, view important information such as menus and availability, and book tables in advance. By providing a platform for restaurant discovery and booking, the system improves the dining experience for users while helping restaurants manage reservations more efficiently.

1.1 Document Purpose

The purpose of this Software Requirements Specification (SRS) document is to describe the requirements of the Restaurant Reservation System. This document provides a comprehensive overview of the system's functionality, design constraints, and performance expectations.

The scope of this SRS includes the functional and non-functional requirements for the core features of the system, including user management, restaurant discovery, table availability management, and reservation handling. The document focuses on the overall system architecture and database interactions that support the reservation process. The system will be implemented using a **React-based frontend, a Node.js backend, and a PostgreSQL database**, providing a scalable platform for restaurant discovery and reservation services.

1.2 Product Scope

The Restaurant Reservation System is a full-stack web application that allows users to discover restaurants and make reservations online. The system enables customers to search for restaurants, view restaurant details such as menus and operating hours, check table availability, and book reservations for specific dates and times.

The primary objective of the system is to simplify the reservation process and prevent issues such as overbooking by implementing validation rules within the system. The platform also provides an intuitive user interface to improve user experience and make restaurant discovery easier. This system benefits both customers and restaurants by providing a reliable and efficient reservation management platform.

1.3 Intended Audience and Document Overview

This document is intended for several groups involved in the development and evaluation of the system.

*The primary audience includes **software developers**, who will use this document to understand the technical requirements and system functionality. **Project evaluators and instructors** will use the document to assess whether the project meets the required specifications. Additionally, **team members and project stakeholders** may refer to this document to understand the system design and expected outcomes.*

The remainder of this document is organized as follows. The next sections describe the overall system architecture, system features, external interfaces, database requirements, and

non-functional requirements. These sections collectively provide a detailed understanding of how the system is expected to operate.

1.4 Definitions, Acronyms and Abbreviations

API – Application Programming Interface used for communication between the frontend and backend.

DB – Database used to store system data such as users, restaurants, and reservations.

Frontend – The user interface layer of the system developed using React.

Backend – The server-side logic responsible for handling requests and communicating with the database.

Reservation – The process of booking a table at a restaurant for a specific date and time.

UI – User Interface through which users interact with the system.

UX – User Experience referring to the overall usability and interaction quality of the system.

SRS – Software Requirements Specification document that describes system requirements.

1.5 Document Conventions

This document follows the IEEE recommended structure for Software Requirements Specification documents. The document is organized into sections and subsections to clearly present the system requirements.

Standard formatting conventions have been followed throughout the document. Section titles are numbered to maintain a clear hierarchy of information. Technical terms and system components are written consistently to avoid ambiguity. Descriptions are written in clear and concise language to ensure that the requirements are easily understood by both technical and non-technical readers.

The document uses a structured format so that each requirement can be easily referenced during the development and testing phases of the project.

1.6 References and Acknowledgments

The following documents, standards, and resources were referred to during the preparation of this Software Requirements Specification (SRS):

1. **Statement of Work (SOW)** – Restaurant Reservation & Discovery System, Mahindra University Software Engineering Course, 2026.
2. **React Documentation**

<https://react.dev>

Official documentation used as reference for frontend development.

3. FastApi Documentation

<https://fastapi.tiangolo.com/>

Used as a reference for backend server implementation.

4. PostgreSQL Documentation

<https://www.postgresql.org/docs/>

Used for database design and query implementation.

2 Overall Description

2.1 Product Overview

The Restaurant Reservation & Discovery System is a web-based application designed to simplify the process of discovering restaurants and making table reservations. The system allows users to browse restaurants, view menus and availability, and reserve tables online without needing to contact the restaurant directly.

*This product is a **new standalone system** designed to improve convenience for customers while helping restaurants manage their reservations efficiently. The system will be developed as a full-stack web application using **React for the frontend, FastApi for the backend, and PostgreSQL for the database.***

2.2 Product Functionality

The system will provide the following major functionalities:

- *User registration and login authentication*
- *Searching and discovering restaurants*
- *Viewing restaurant details such as menu, location, and operating hours*
- *Checking restaurant table availability*
- *Making table reservations for specific dates and times*
- *Canceling or modifying reservations*
- *Preventing overbooking through validation rules*
- *Admin interface for managing restaurant listings*
- *Admin ability to update restaurant menus and availability*
- *Storing and retrieving reservation data from the database*

These functionalities provide users with a convenient platform for discovering restaurants and managing reservations efficiently.

2.3 Design and Implementation Constraints

The development of the Restaurant Reservation System is subject to several constraints that influence its design and implementation.

*The system must be developed using the technologies specified in the project scope. The frontend will be implemented using **React**, while the backend will be developed using **Fast API**. The system database will use **PostgreSQL** for data storage and management.*

The application must operate within standard web browsers and rely on internet connectivity for communication between the frontend, backend, and database. The design must also follow standard web development practices to ensure compatibility across major browsers.

Secure handling of user data and authentication mechanisms must be implemented. Additionally, the system must be developed within the timeline and milestones specified in the project plan.

2.4 Assumptions and Dependencies

Several assumptions and dependencies affect the development and operation of the system.

It is assumed that restaurants participating in the platform will provide accurate information about their menus, operating hours, and table availability.

The system depends on several external technologies and tools, including React for frontend development, FastApi for backend services, and PostgreSQL for database management. Proper functioning of these technologies is essential for the successful operation of the system.

Additionally, the project assumes that all team members will collaborate effectively to complete frontend, backend, and database development within the defined project timeline.

3 Specific Requirements

3.1 External Interface Requirements

3.1.1 User Interfaces

The system will provide a web-based user interface that allows users to interact with the Restaurant Reservation & Discovery System through a web browser. The interface will be designed to be simple, intuitive, and responsive so that users can easily search for restaurants and make reservations.

The main user interface components include:

- **Home Page** – Displays featured restaurants and a search bar for discovering restaurants.
- **Search Page** – Allows users to search restaurants based on name, cuisine, or location.
- **Restaurant Details Page** – Shows restaurant information such as menu, operating hours, ratings, and available reservation slots.
- **Reservation Page** – Allows users to select date, time, and number of guests to make a reservation.
- **User Dashboard** – Displays user profile information and reservation history.
- **Admin Dashboard** – Allows administrators to manage restaurant listings, menus, and reservations.

*The interface will be developed using **React** and will follow standard web design practices to ensure compatibility across modern web browsers.*

3.1.2 Hardware Interfaces

The system does not require specialized hardware components. Users will interact with the system using standard computing devices such as:

- Desktop computers
- Laptops
- Tablets
- Smartphones (through web browser)

These devices will communicate with the system through an internet connection using standard web protocols. No additional sensors or specialized hardware interfaces are required for the system.

3.1.3 Software Interfaces

The system will interact with several software components to function properly.

- **Frontend Interface:**
The user interface will be implemented using React, which communicates with the backend server through RESTful APIs.
- **Backend Server:**
The backend will be developed using FastAPI to handle user requests, reservation logic, and communication with the database.
- **Database System:**
PostgreSQL will be used as the database management system to store and manage data related to users, restaurants, menus, and reservations.
- **Web Browser:**
Users will access the system using modern web browsers such as Google Chrome, Safari, Microsoft Edge, or Firefox.

These software components work together to ensure smooth communication between the user interface, application logic, and database.

3.2 Functional Requirements

3.2.1 F1: User Registration

The system shall allow new users to create an account by providing basic information such as name, email address, and password.

3.2.2 F2: User Login

The system shall allow registered users to log in securely using their email and password.

3.2.3 F3: Restaurant Search

The system shall allow users to search for restaurants using keywords such as restaurant name, cuisine type, or location.

3.2.4 F4: View Restaurant Details

The system shall allow users to view restaurant information including menu items, operating hours, ratings, and location.

3.2.5 F5: Check Table Availability

The system shall allow users to check available reservation slots for a selected restaurant based on date, time, and number of guests.

3.2.6 F6: Make Reservation

The system shall allow users to reserve a table by selecting a restaurant, date, time, and number of guests.

3.2.7 F7: Cancel Reservation

The system shall allow users to cancel previously made reservations.

3.2.8 F8: View Reservation History

The system shall allow users to view their past and upcoming reservations through their user dashboard.

3.2.9 F9: Admin Restaurant Management

The system shall allow administrators to add, edit, or remove restaurant listings from the system.

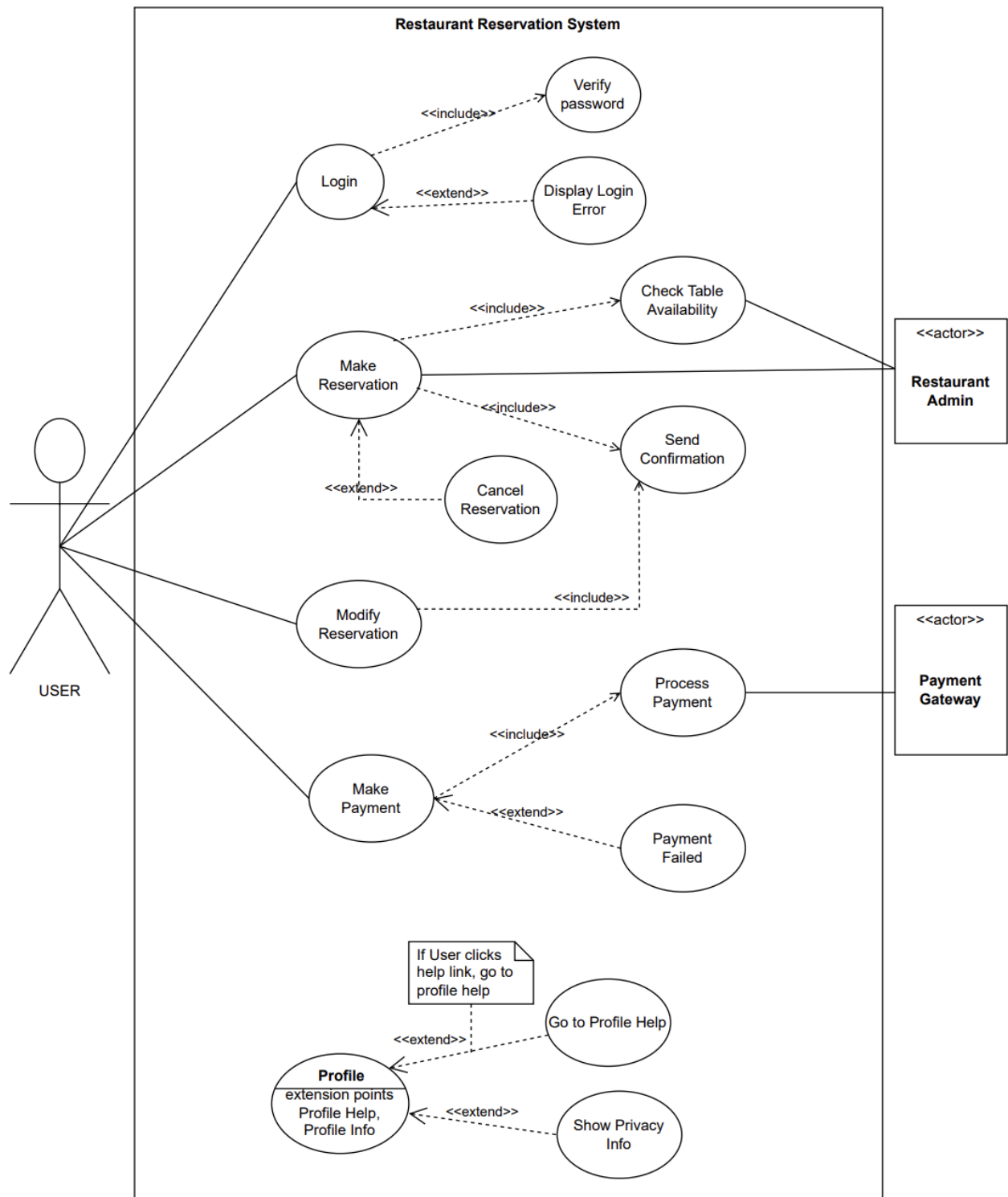
3.2.10 F10: Menu Management

The system shall allow administrators to update restaurant menu items and related information.

3.2.11 F11: Reservation Validation

The system shall implement validation rules to prevent overbooking and ensure that reservations are made only when tables are available.

3.3 Use Case Model



3.3.1 Use Case #1 - Login

Author – Charith Leburu

Purpose - The purpose of this use case is to allow a registered user to securely access the Restaurant Reservation System by verifying their credentials. Successful login enables the user to access system features such as making reservations, modifying bookings, and managing their profile

Requirements Traceability :

- User Registration
- User Login
- View Reservation History

Priority - High, Login functionality is critical because users must be authenticated before accessing reservation and account features.

Preconditions -

- The user must already have a registered account in the system.
- The system database must be available.
- The user must have internet access and access to a supported web browser.

Post conditions -

Successful Completion:

- The user is authenticated and redirected to the user dashboard.

Failure:

- An error message is displayed indicating incorrect login credentials.

Actors –

- Primary Actor: User
- Supporting Actor: System (authentication service)

Extends –

- Display Login Error

Flow of Events

1. Basic Flow

The user opens the Restaurant Reservation System.

The user selects the Login option.

The system displays the login form.
The user enters their email and password.
The system verifies the password.
The system authenticates the user credentials.
The user is successfully logged into the system.
The system redirects the user to the dashboard.

2. Alternative Flow

A1: Incorrect Password

The user enters incorrect login credentials.
The system fails to verify the password.
The system displays an error message indicating invalid credentials.
The user is prompted to try logging in again.

3. Exceptions

- E1: System Database Unavailable:** If authentication is unavailable, the system displays a system error message and prevents login attempts
- E2: Network Connection Failure:** If the user loses internet connectivity, the login process cannot be completed.

Includes (Other Use Case IDs)

- Verify Password

Notes / Issues

- Passwords must be securely stored using hashing algorithms.
- The system should implement security measures such as protection against repeated failed login attempts.
- Future versions of the system may include features such as multi-factor authentication (MFA).

3.3.2 Use Case #2 - Make Reservation (U2)

Author – Charith Leburu

Purpose - The purpose of this use case is to allow a user to reserve a table at a restaurant for a selected date, time, and number of guests. The system checks table availability before confirming the reservation.

Requirements Traceability :

- Check Table Availability
- Make Reservation
- Reservation Validation

Priority - High, Reservations are the core functionality of the system and must work reliably.

Preconditions -

- The user must be logged into the system.
- The selected restaurant must exist in the system.
- The restaurant must have available tables.

Post conditions -

Successful Completion:

- The reservation is stored in the database.
- The user receives a reservation confirmation.

Failure:

- The system informs the user that the requested reservation slot is unavailable.

Actors –

- Primary Actor: User
- Supporting Actor: Restaurant Admin (provides table availability data)

Extends –

- Cancel Reservation

Flow of Events

2. Basic Flow

The user searches for a restaurant.

The user selects a restaurant.

The system displays restaurant details and reservation options.

The user selects the reservation date, time, and number of guests.

The system checks table availability.

If tables are available, the user confirms the reservation.

The system records the reservation in the database.

The system sends a reservation confirmation to the user.

3. Alternative Flow

A1: No Tables Available

The system checks table availability.

No tables are available for the selected time.
The system informs the user.
The user selects another time slot.

4. Exceptions

•**E1: Database Error:** If the system cannot store the reservation due to a database error, the reservation is not completed.

•**E2: Network Connection Failure:** If the network connection fails during the reservation process, the request is cancelled.

Includes (Other Use Case IDs)

- Check Table Availability
- Send Confirmation

Notes / Issues

- The system should prevent duplicate bookings.
- Reservation validation rules must prevent overbooking.

3.3.3 Use Case #3 - Modify Reservation (U3)

Author – Charith Leburu

Purpose - The purpose of this use case is to allow users to update an existing reservation by changing the reservation time, date, or number of guests.

Requirements Traceability :

- Make Reservation
- View Reservation History

Priority - Medium, Reservation modification improves user convenience but is not critical for initial system operation.

Preconditions -

- The user must be logged into the system.
- The user must already have an existing reservation.

Post conditions -

Successful Completion:

- The reservation details are updated in the system.

Failure:

- The reservation remains unchanged.

Actors –

- Primary Actor: User

Extends –

- None

Flow of Events

5. Basic Flow

The user logs into the system.

The user opens the reservation dashboard.

The user selects an existing reservation.

The user chooses the Modify Reservation option.

The user updates reservation details such as date, time, or number of guests.

The system checks table availability for the updated request.

If available, the system updates the reservation.

The system sends an updated confirmation to the user.

6. Alternative Flow

A1: New Time Slot Unavailable

The user modifies reservation details.

The system checks availability.

No tables are available for the new request.

The system prompts the user to choose another slot.

7. Exceptions

•E1: **Reservation Not Found:** If the reservation record cannot be found in the database, the system displays an error.

Includes (Other Use Case IDs)

- Send Confirmation

Notes / Issues

- Modification may be restricted close to reservation time depending on restaurant policy.

3.3.4 Use Case #4 - Make Payment (U4)

Author – Charith Leburu

Purpose - The purpose of this use case is to allow the user to complete payment for a reservation using an external payment gateway.

Requirements Traceability :

- Payment Processing

Priority - Medium, Payments may be required depending on restaurant policy.

Preconditions -

- The user must have a confirmed reservation.
- The user must be logged into the system.
- The payment gateway must be available.

Post conditions -

Successful Completion:

- Payment is successfully processed.
- The reservation status is updated as paid.

Failure:

- The payment is declined and the reservation remains unpaid.

Actors –

- Primary Actor: User
- Supporting Actor: Payment Gateway

Extends –

- Payment Failed

Flow of Events

8. Basic Flow

The user selects the Make Payment option.

The system redirects the user to the payment interface.

The user enters payment details.

The system sends the payment request to the payment gateway.

The payment gateway processes the transaction.

If successful, the system records the payment.

The system updates the reservation status.

The user receives payment confirmation.

9. Alternative Flow

A1: Payment Declined

The payment gateway declines the transaction.

The system informs the user that the payment failed.

The user can retry payment.

10. Exceptions

- **E1: Payment Gateway Unavailable:** If the payment service is unavailable, the payment cannot be processed.

Includes (Other Use Case IDs)

- Process Payment

Notes / Issues

- Payment information must be handled securely.
- The system should support retry attempts for failed payments.

4 Other Non-functional Requirements

4.1 Performance Requirements

The Restaurant Reservation System must meet the following performance requirements to ensure smooth user experience and reliable system operation.

- P1: The system shall display restaurant search results within 2 seconds after a user submits a search query.
- P2: The system shall confirm a reservation request within 3 seconds after the user submits reservation details.
- P3: The system shall support multiple concurrent users accessing the system without significant performance degradation.
- P4: The system shall retrieve restaurant information, menus, and reservation details from the database within 2 seconds under normal operating conditions.

- P5: Payment processing through the payment gateway should complete within 5 seconds, depending on the response time of the external payment service.

4.2 Safety and Security Requirements

The system must ensure the security of user data and protect the system from unauthorized access.

- *S1: The system shall require users to authenticate using a valid email and password before accessing reservation features.*
- *S2: User passwords shall be securely stored in the database using hashed encryption methods.*
- *S3: The system shall prevent unauthorized users from accessing or modifying reservation data.*
- *S4: All communication between the client (web browser) and the server shall use secure HTTPS connections.*
- *S5: Payment information shall be processed only through a secure external payment gateway, and the system shall not store sensitive payment details.*
- *S6: The system shall implement validation rules to prevent duplicate reservations and potential misuse of the reservation system.*

4.3 Software Quality Attributes

4.3.1 Reliability

The system must operate reliably to ensure that users can successfully make and manage reservations.

- The system shall maintain accurate reservation records in the database.
- The system shall prevent overbooking by validating table availability before confirming reservations.
- In case of system errors, appropriate error messages shall be displayed to users.

4.3.2 Usability

The system should be easy to use and accessible to users with basic technical knowledge.

- The user interface shall be simple and intuitive.
- Users should be able to complete a reservation in no more than five steps.
- Navigation between pages should be clear and consistent.

4.3.3 Maintainability

The system should be designed in a way that allows future updates and modifications.

- The application shall follow a modular architecture separating frontend, backend, and database components.
- Developers should be able to update or extend system features without affecting existing functionality.

4.3.4 Availability

The system should be available for users at most times.

- The system shall maintain high availability to allow users to access restaurant information and make reservations anytime.
- Scheduled maintenance should be announced in advance.

Appendix A – Data Dictionary

Data Item	Type	Description	Example
UserID	Integer / UUID	Unique identifier for each registered user	1
Name	String	Full name of the user	Aryan Goud
Email	String	Email address used for login	user@email.com
PasswordHash	String	Encrypted password stored in database	Hashed value
RestaurantID	Integer	Unique identifier for each restaurant	2
RestaurantName	String	Name of the restaurant	Paul
CuisineType	String	Type of food served	Italian
ReservationID	Integer	Unique identifier for reservation	9001
ReservationDate	Date	Date of the reservation	2026-03-20
ReservationTime	Time	Time slot selected	7:30 PM
NumberOfGuests	Integer	Number of people for reservation	4
ReservationStatus	Enum	Status of reservation	Confirmed / Cancelled

Appendix B - Group Log

Meeting	Date	Members Present	Discussion	Outcome
1	Feb 5	All members	Initial Project discussion	Introduced all members of the team to each other
2	Feb 13	All members	Discussed system features and requirements	created a draft on system requirements doc
3	Feb 20	All members	Worked on system architecture diagram	created system arch diagram and finalized on stack
4	Mar 14	All members	Worked on SRS document	finalized SRS document